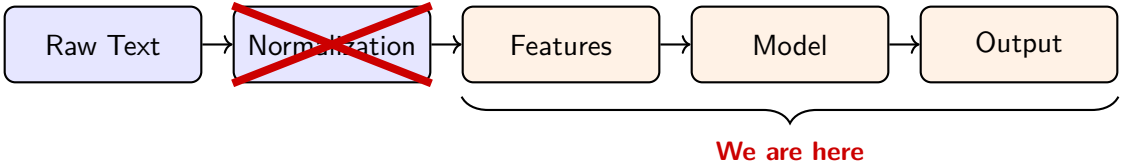
# Textual Analysis in Finance

## Week 5: Large Language Models

Tri, Minh Phan

Faculty of Business and Economics
University of Basel

# Where are we?



# Outline

# From LMs to LLMs

# Why do language models get bigger?

- By 2019-2020, empirical evidence suggested that **larger models consistently outperform smaller ones**
  - $\text{BERT}_{\text{LARGE}}$ (340M parameters) outperforms $\text{BERT}_{\text{BASE}}$ (110M parameters) across a wide range of tasks (e.g., sentiment analysis, question answering, natural language inference) [Devlin et al., 2019]
  - Larger variants of GPT-3 (e.g., 760M parameters) substantially outperform smaller versions (125M, 356M) on many benchmarks [Brown et al., 2020]

# Why do language models get bigger?

- By 2019-2020, empirical evidence suggested that **larger models consistently outperform smaller ones**
  - $\text{BERT}_{\text{LARGE}}$ (340M parameters) outperforms $\text{BERT}_{\text{BASE}}$ (110M parameters) across a wide range of tasks (e.g., sentiment analysis, question answering, natural language inference) [Devlin et al., 2019]
  - Larger variants of GPT-3 (e.g., 760M parameters) substantially outperform smaller versions (125M, 356M) on many benchmarks [Brown et al., 2020]
- **Performance improvements were systematic**: test loss decreases with increases in model size, data size, and compute [Kaplan et al., 2020]

# Why do language models get bigger?

- By 2019-2020, empirical evidence suggested that **larger models consistently outperform smaller ones**
  - $\text{BERT}_{\text{LARGE}}$ (340M parameters) outperforms $\text{BERT}_{\text{BASE}}$ (110M parameters) across a wide range of tasks (e.g., sentiment analysis, question answering, natural language inference) [Devlin et al., 2019]
  - Larger variants of GPT-3 (e.g., 760M parameters) substantially outperform smaller versions (125M, 356M) on many benchmarks [Brown et al., 2020]
- **Performance improvements were systematic**: test loss decreases with increases in model size, data size, and compute [Kaplan et al., 2020]
- This naturally led to aggressive scaling along three dimensions:
  - **Model size:** Millions $\rightarrow$ billions (and beyond) of parameters
  - **Training data:** Gigabytes $\rightarrow$ terabytes $\rightarrow$ internet-scale corpora
  - **Compute:** Massive distributed GPU/TPU clusters enabling large-scale optimization

# Computational cost: The binding constraint

When language models get bigger, ...

- **Training costs grow superlinearly**
  - Larger models require exponentially more computing power, which costs tens to hundreds of millions of dollars

# Computational cost: The binding constraint

When language models get bigger, ...

- **Training costs grow superlinearly**
  - Larger models require exponentially more computing power, which costs tens to hundreds of millions of dollars
- **Data curation becomes industrial-scale**
  - Web-scale scraping, filtering, de-duplication, etc.
  - Data quality directly affects scaling efficiency

# Computational cost: The binding constraint

When language models get bigger, ...

- **Training costs grow superlinearly**
  - Larger models require exponentially more computing power, which costs tens to hundreds of millions of dollars
- **Data curation becomes industrial-scale**
  - Web-scale scraping, filtering, de-duplication, etc.
  - Data quality directly affects scaling efficiency
- **Infrastructure requirements escalate**
  - Massive distributed GPU/TPU clusters
  - Specialized hardware (e.g., A100/H100-class accelerators)
  - Energy consumption becomes economically and politically relevant

# Computational cost: The binding constraint

When language models get bigger, ...

- **Training costs grow superlinearly**
  - Larger models require exponentially more computing power, which costs tens to hundreds of millions of dollars
- **Data curation becomes industrial-scale**
  - Web-scale scraping, filtering, de-duplication, etc.
  - Data quality directly affects scaling efficiency
- **Infrastructure requirements escalate**
  - Massive distributed GPU/TPU clusters
  - Specialized hardware (e.g., A100/H100-class accelerators)
  - Energy consumption becomes economically and politically relevant

→ Scaling is no longer purely a modeling question, it becomes an **economic one**

# Scaling changes how we adapt and use language models

- In the pre-LLM era (e.g., BERT, early GPT), models were **fully fine-tuned** for each downstream task:
  - One BERT model for sentiment analysis
  - Another BERT model for topic classification
  - Another for question answering

# Scaling changes how we adapt and use language models

- In the pre-LLM era (e.g., BERT, early GPT), models were **fully fine-tuned** for each downstream task:
  - One BERT model for sentiment analysis
  - Another BERT model for topic classification
  - Another for question answering
- At the LLM scale, full fine-tuning becomes computationally costly

# Scaling changes how we adapt and use language models

- In the pre-LLM era (e.g., BERT, early GPT), models were **fully fine-tuned** for each downstream task:
  - One BERT model for sentiment analysis
  - Another BERT model for topic classification
  - Another for question answering
- At the LLM scale, full fine-tuning becomes computationally costly
- Instead, practitioners train a single large model and adapt it via **prompting** and **lightweight adaptation methods**

# Scaling changes how we adapt and use language models

- In the pre-LLM era (e.g., BERT, early GPT), models were **fully fine-tuned** for each downstream task:
  - One BERT model for sentiment analysis
  - Another BERT model for topic classification
  - Another for question answering
- At the LLM scale, full fine-tuning becomes computationally costly
- Instead, practitioners train a single large model and adapt it via **prompting** and **lightweight adaptation methods**
- The objective shifts: From producing numerical embeddings to generating coherent, task-completing texts

# Scaling changes how we adapt and use language models

- In the pre-LLM era (e.g., BERT, early GPT), models were **fully fine-tuned** for each downstream task:
  - One BERT model for sentiment analysis
  - Another BERT model for topic classification
  - Another for question answering
- At the LLM scale, full fine-tuning becomes computationally costly
- Instead, practitioners train a single large model and adapt it via **prompting** and **lightweight adaptation methods**
- The objective shifts: From producing numerical embeddings to generating coherent, task-completing texts

→ The view of LMs shifts from **representation learning** to a **general cognitive interface** (or text generation)

# Training an LLM

# Reinforcement Learning from Human Feedback (RLHF)

*Making language models bigger does not inherently make them better at following a user's intent [Ouyang et al., 2022]*

- General-purpose cognitive interfaces require **more guided feedback**

# Reinforcement Learning from Human Feedback (RLHF)

*Making language models bigger does not inherently make them better at following a user's intent [Ouyang et al., 2022]*

- General-purpose cognitive interfaces require **more guided feedback**
- Pre-LLM models (e.g., BERT, XLNet, GPT) are trained purely on text data:
  - They generate text statistically, without understanding intent
  - As a result, outputs can be untruthful, toxic, or unhelpful

# Reinforcement Learning from Human Feedback (RLHF)

*Making language models bigger does not inherently make them better at following a user's intent [Ouyang et al., 2022]*

- General-purpose cognitive interfaces require **more guided feedback**
- Pre-LLM models (e.g., BERT, XLNet, GPT) are trained purely on text data:
  - They generate text statistically, without understanding intent
  - As a result, outputs can be untruthful, toxic, or unhelpful
- RLHF provides **human feedback to guide the model** toward helpful, safe, and aligned responses

# RLHF - Analogy to raising kids

- **Pre-training = Early-life learning**
  - Raising kids: The kid learns from everything they observe (books, conversations, TV) without guidance
  - RLHF analogy: LLMs learn patterns in text data statistically. This is similar to training GPT or BERT

# RLHF - Analogy to raising kids

- **Pre-training = Early-life learning**
  - Raising kids: The kid learns from everything they observe (books, conversations, TV) without guidance
  - RLHF analogy: LLMs learn patterns in text data statistically. This is similar to training GPT or BERT
- **Uncontrolled behavior = Missteps**
  - Raising kids: Without guidance, the kid might repeat mistakes, tell untruths, or misunderstand social norms
  - RLHF analogy: LLMs can produce toxic, unhelpful, or incorrect responses

# RLHF - Analogy to raising kids

- **Pre-training = Early-life learning**
  - Raising kids: The kid learns from everything they observe (books, conversations, TV) without guidance
  - RLHF analogy: LLMs learn patterns in text data statistically. This is similar to training GPT or BERT
- **Uncontrolled behavior = Missteps**
  - Raising kids: Without guidance, the kid might repeat mistakes, tell untruths, or misunderstand social norms
  - RLHF analogy: LLMs can produce toxic, unhelpful, or incorrect responses
- **Human feedback = Guidance & correction**
  - Raising kids: Parents provide feedback, rewards, and corrections to teach the kid what is safe, helpful, and appropriate
  - RLHF analogy: humans rate outputs, guiding the model to align with human intent

# RLHF - Analogy to raising kids

- **Pre-training = Early-life learning**
  - Raising kids: The kid learns from everything they observe (books, conversations, TV) without guidance
  - RLHF analogy: LLMs learn patterns in text data statistically. This is similar to training GPT or BERT
- **Uncontrolled behavior = Missteps**
  - Raising kids: Without guidance, the kid might repeat mistakes, tell untruths, or misunderstand social norms
  - RLHF analogy: LLMs can produce toxic, unhelpful, or incorrect responses
- **Human feedback = Guidance & correction**
  - Raising kids: Parents provide feedback, rewards, and corrections to teach the kid what is safe, helpful, and appropriate
  - RLHF analogy: humans rate outputs, guiding the model to align with human intent
- **Outcome = Aligned behavior**
  - Raising kids: With consistent feedback, the kid learns to behave thoughtfully
  - RLHF analogy: LLMs trained with RLHF generate responses that are truthful, helpful, and aligned with user expectations

# Let's dive a bit deeper

RLHF comprises the following steps [Ouyang et al., 2022]:

- Train an LLM, e.g., GPT-3, with the whole internet

# Let's dive a bit deeper

RLHF comprises the following steps [Ouyang et al., 2022]:

- Train an LLM, e.g., GPT-3, with the whole internet
- Collect human feedback data and train the LLM on it
  1. Hire a human team to generate feedback on available prompts
  2. Fine-tune the LLM on the human-based prompt-feedback dataset (called **supervised fine-tuning**)

# Let's dive a bit deeper

RLHF comprises the following steps [Ouyang et al., 2022]:

- Train an LLM, e.g., GPT-3, with the whole internet
- Collect human feedback data and train the LLM on it
  1. Hire a human team to generate feedback on available prompts
  2. Fine-tune the LLM on the human-based prompt-feedback dataset (called **supervised fine-tuning**)
- Collect comparison data and train a reward model
  1. Collect more prompts and the corresponding LLM's responses (several responses per prompt)
  2. Ask humans again to rank the preference of those responses
  3. Use those prompts, responses, and human ranking to train another small LM, called the **reward model**
  4. The reward model outputs a score per response (no text generation), in which more preferred responses receive higher scores

# Let's dive a bit deeper

- Reinforcement learning happens here:
  1. Feed new prompts to the LLM
  2. The LLM generates (usually) one response per prompt
  3. The reward model scores the responses
  4. The LLM is updated using **Proximal Policy Optimization (PPO)** to increase the probability of higher-reward responses
  5. Repeat Steps 2-4 until

# Let's dive a bit deeper

- Reinforcement learning happens here:
  1. Feed new prompts to the LLM
  2. The LLM generates (usually) one response per prompt
  3. The reward model scores the responses
  4. The LLM is updated using **Proximal Policy Optimization (PPO)** to increase the probability of higher-reward responses
  5. Repeat Steps 2-4 until
- PPO pushes the LLM toward higher-reward responses **gradually** (i.e., it does not move "far" from the previous version)
  - Like teaching a robot to cook for someone who wants to gain weight
  - If we force the robot to get it perfect in one step, it might just hand us a bottle of cooking oil
  - PPO forces the robot to learn gradually: make a first dish, tweak the salt or sugar, get feedback, repeat step by step

# Text generation

# How does an LLM generate texts?

- Language modeling consists of estimating the conditional probability $p(w_{t+1} \mid w_{1:t})$ using deep neural networks (*language models*)

# How does an LLM generate texts?

- Language modeling consists of estimating the conditional probability $p(w_{t+1} \mid w_{1:t})$ using deep neural networks (*language models*)
- Intuition: given a sequence of tokens $w_{1:t}$ (the *context*), the model predicts a distribution over the next token $w_{t+1}$

# How does an LLM generate texts?

- Language modeling consists of estimating the conditional probability $p(w_{t+1} \mid w_{1:t})$ using deep neural networks (*language models*)
- Intuition: given a sequence of tokens $w_{1:t}$ (the *context*), the model predicts a distribution over the next token $w_{t+1}$
- A simple decoding rule is to choose the most likely token: $w_{t+1} = \arg\max_w \hat{p}(w \mid w_{1:t})$

# How does an LLM generate texts?

- Language modeling consists of estimating the conditional probability $p(w_{t+1} \mid w_{1:t})$ using deep neural networks (*language models*)
- Intuition: given a sequence of tokens $w_{1:t}$ (the *context*), the model predicts a distribution over the next token $w_{t+1}$
- A simple decoding rule is to choose the most likely token: $w_{t+1} = \arg\max_w \hat{p}(w \mid w_{1:t})$
- Text generation proceeds *iteratively*: after generating $w_{t+1}$, it is appended to the context and the process repeats

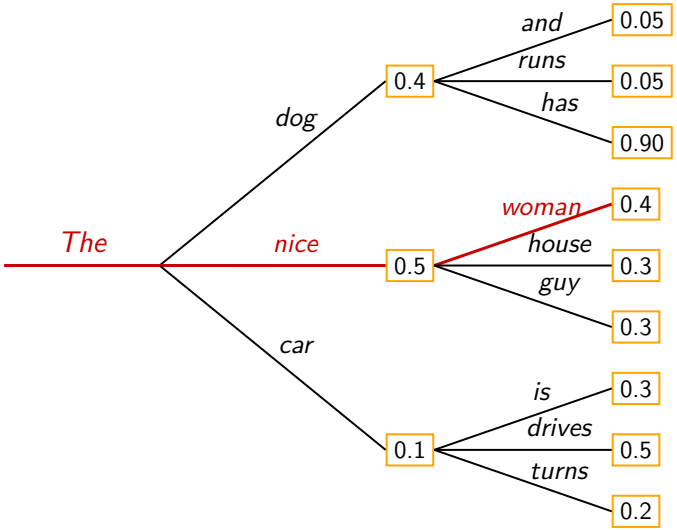
# How does an LLM generate texts?

- Language modeling consists of estimating the conditional probability $p(w_{t+1} \mid w_{1:t})$ using deep neural networks (*language models*)
- Intuition: given a sequence of tokens $w_{1:t}$ (the *context*), the model predicts a distribution over the next token $w_{t+1}$
- A simple decoding rule is to choose the most likely token: $w_{t+1} = \arg\max_w \hat{p}(w \mid w_{1:t})$
- Text generation proceeds *iteratively*: after generating $w_{t+1}$, it is appended to the context and the process repeats
- **Is it really that simple?** In practice, different *decoding strategies* (sampling, temperature, top-$k$, top-$p$) are used to generate more diverse and natural text

# How does an LLM generate texts?

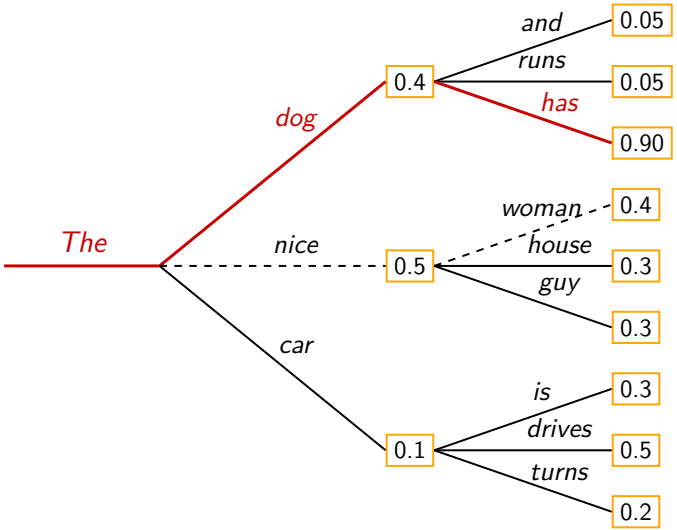
- Language modeling consists of estimating the conditional probability $p(w_{t+1} \mid w_{1:t})$ using deep neural networks (*language models*)
- Intuition: given a sequence of tokens $w_{1:t}$ (the *context*), the model predicts a distribution over the next token $w_{t+1}$
- A simple decoding rule is to choose the most likely token: $w_{t+1} = \arg\max_w \hat{p}(w \mid w_{1:t})$
- Text generation proceeds *iteratively*: after generating $w_{t+1}$, it is appended to the context and the process repeats
- **Is it really that simple?** In practice, different *decoding strategies* (sampling, temperature, top-$k$, top-$p$) are used to generate more diverse and natural text
- A great blog about text generation can be found here

# Greedy search



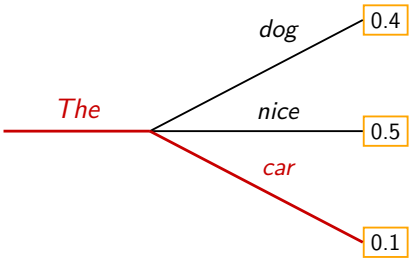

- Greedy search, a.k.a Just go for the one with the **highest probability**
- Start with "The" $\rightarrow$ "nice" $\rightarrow$ "woman"
- Main drawback: *it misses high probability words hidden behind a low probability word*
- The probability of the whole generated text is $0.5 \times 0.4$
- "The" $\rightarrow$ "dog" $\rightarrow$ "has" is a better choice!

# Beam search



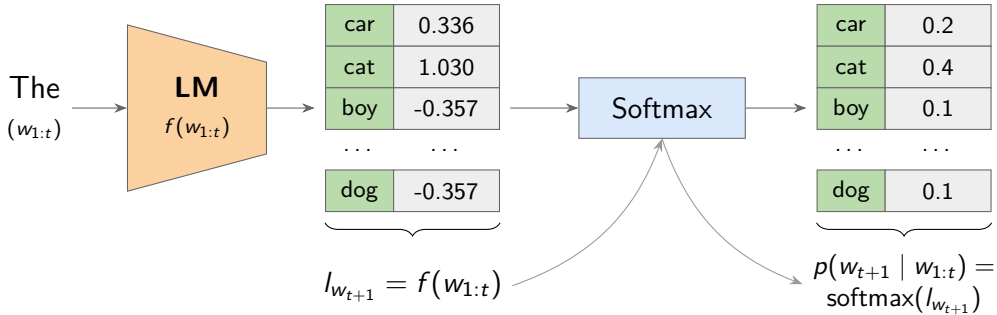

- The probability of the full sequence can be higher than in greedy search: $0.4 \times 0.9 > 0.5 \times 0.4$
- In this case, beam width $= 2$ (number of candidate sequences kept at each step)
- Controlled by `num_beams` in the `.generate()` method
- Drawbacks: Generated texts often ...
  - contain repetitions
  - lack of diversity [Holtzman et al., 2019]

# Sampling



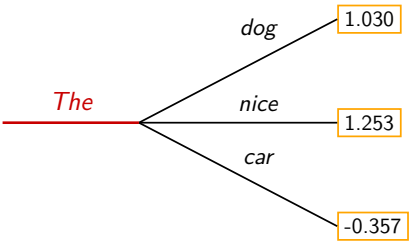

- Purpose: introduce stochasticity in text generation
- Instead of choosing $w_{t+1} = \arg\max_w \hat{p}(w \mid w_{1:t})$, sample the next token $w_{t+1} \sim \hat{p}(w \mid w_{1:t})$
- Low-probability tokens can occasionally be selected (e.g., "car") $\rightarrow$ more diverse and creative text
- The randomness is controlled by `set_seed` and `do_sample`
- Drawback: Generated texts can be often gibberish, meaning, they don't make sense

# Compute $p(w_{t+1}|w_{1:t})$




- Given the context $w_{1:t}$, an LM will compute "logits" of the next token, $l_{w_{t+1}}$
- Basically, each token in the vocabulary has a logit value
- Softmax is a function that transforms logits, a list of numbers, into probabilities, i.e., also a list of numbers, but bounded between 0 and 1, and sum up to 1

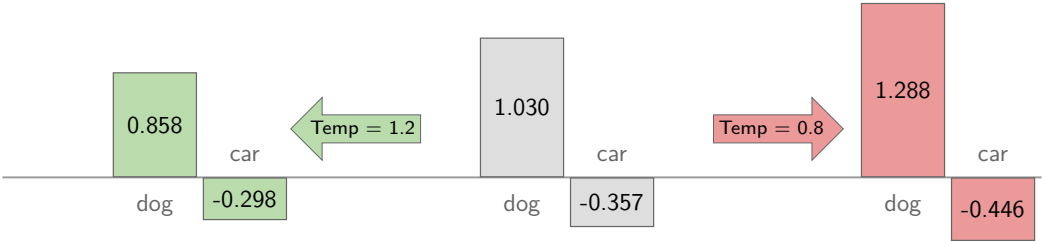
# Sampling - Temperature




How does Softmax work in this case?

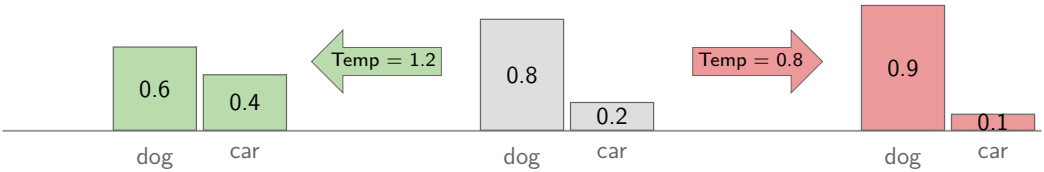
- Assume the vocabulary contains only 3 tokens: dog, nice, and car
- Softmax first computes $e^{1.030}$, $e^{1.253}$, and $e^{-0.357}$ for each word
- Then, $p(w_{t+1} = \text{dog}|w_{1:t}) = \frac{e^{1.030}}{e^{1.030}+e^{1.253}+e^{-0.357}} \approx 0.4$
- Or, $p(w_{t+1} = \text{dog}|w_{1:t}) = \frac{e^{\frac{1.030}{\mathbf{1}}}}{e^{\frac{1.030}{\mathbf{1}}}+e^{\frac{1.253}{\mathbf{1}}}+e^{\frac{-0.357}{\mathbf{1}}}}$
- The $\mathbf{1}$'s are called **Temperature**, and used to control the distribution of the next token
- The name "Temperature" comes from Boltzmann distribution in thermodynamics

# The effect of "Temperature"




- Temperature scales the logits and ...

# The effect of "Temperature"




- Temperature scales the logits and **changes the spread of probabilities**
- **High temperature** → smaller logit differences → flatter distribution
- **Low temperature** → larger logit differences → shaper distribution

# How does Temperature affect text generation?

- Reminder: text is generated by sampling from $p(w_{t+1} \mid w_{1:t})$

# How does Temperature affect text generation?

- Reminder: text is generated by sampling from $p(w_{t+1} \mid w_{1:t})$
- **High temperature** $\rightarrow$ flatter $p(w_{t+1} \mid w_{1:t})$
  - $\rightarrow$ Low-probability tokens become more likely
  - $\rightarrow$ Higher chance of selecting diverse tokens
  - $\rightarrow$ **more diverse / creative** outputs

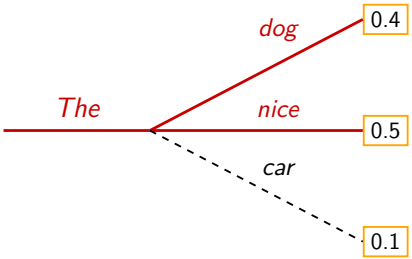
# How does Temperature affect text generation?

- Reminder: text is generated by sampling from $p(w_{t+1} \mid w_{1:t})$
- **High temperature** $\rightarrow$ flatter $p(w_{t+1} \mid w_{1:t})$
  - $\rightarrow$ Low-probability tokens become more likely
  - $\rightarrow$ Higher chance of selecting diverse tokens
  - $\rightarrow$ **more diverse / creative** outputs
- Similarly, **low temperature** $\rightarrow$ shaper $p(w_{t+1}|w_{1:t})$ $\rightarrow$ generated texts are **more conservative** (more deterministic)

# How does Temperature affect text generation?

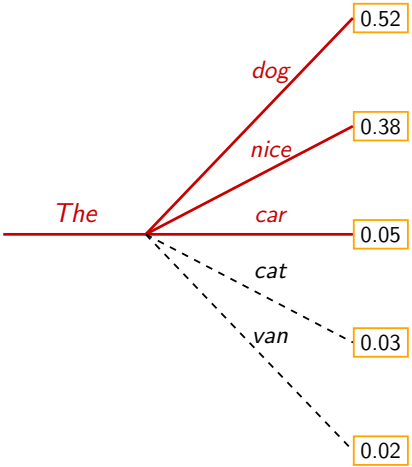
- Reminder: text is generated by sampling from $p(w_{t+1} \mid w_{1:t})$
- **High temperature** $\rightarrow$ flatter $p(w_{t+1} \mid w_{1:t})$
  - $\rightarrow$ Low-probability tokens become more likely
  - $\rightarrow$ Higher chance of selecting diverse tokens
  - $\rightarrow$ **more diverse / creative** outputs
- Similarly, **low temperature** $\rightarrow$ shaper $p(w_{t+1}|w_{1:t}) \rightarrow$ generated texts are **more conservative** (more deterministic)
- More on empirical insights:
  - Temperature is not a "one-size-fits-all" hyperparameter [Li et al., 2025], but DeepSeek recommend some default values depending on the tasks here
  - $T = 0 \rightarrow$ greedy decoding
  - "*...the influence of temperature on creativity is far more nuanced and weak than suggested by the 'creativity parameter' claim...*" [Peeperkorn et al., 2024]
  - Creativity is controlled by many other hyperparameters, not just temperature

# Top-$K$ sampling




- Introduced by Fan et al. [2018], simple yet efficient
- Restrict sampling only from the top $K$ tokens with the highest probabilities
- Probabilities are re-distributed: $(0.5, 0.4) \rightarrow (0.556, 0.444)$
- $K \uparrow \rightarrow$ more diversity; $K \downarrow \rightarrow$ more conservative outputs
- Typically applied after temperature scaling

# Top-$p$ (nucleus) sampling




- Top-$p$ (nucleus) sampling: select the **smallest set of tokens** whose **cumulative probability** $\geq p$
- Example: $p = 0.94 \rightarrow$ keep "dog", "nice", "car" since $0.52 + 0.38 + 0.05 = 0.95$
- Can be combined with Top-$K$:
  - Apply Top-$K$ first, then Top-$p$ source code
  - Final set satisfies **both constraints**

# Why LLMs hallucinate?

- Hallucination: LLMs produce "plausible yet incorrect statements instead of admitting uncertainty" [Kalai et al., 2025]
- LLMs hallucinate, even with the latest model like GPT-5
- But why? Hallucination roots from all steps:
  - Pre-training
  - Calibration by RLHF

# Hallucination from pre-training

- Even with error-free training data, **the statistical objective** minimized during pretraining would lead to an LLM that generates errors
- For example:
  - The sentence "Lionel Messi is a Spanish football player" is correct linguistically, but it's incorrect as a fact
  - The current statistical objectives only care about linguistic correctness, not facts
  - An LLM trained on this data, by the current statistical objective, will hallucinate; i.e., it thinks it is producing a correct response
- Other reasons:
  - Poor models
  - Distribution shift: Users ask questions that have been never seen before in the training data
  - Garbage in, Garbage out: Training data usually contains factual errors as well

# Hallucination from the post-training calibration

- Recall: in RLHF, the reward model (teacher) score the response from the LLM (student) given a prompt
- This motivates guessing when the LLM faces uncertainty
- Analogy:
  - Students are having a multiple choice question in an exam
  - The question has 3 choices: one true, one false, and one "I don't know"
  - We rarely choose "I don't know" as it sounds like an "always wrong" answer $\rightarrow$ it maximally penalize the overall scores
  - It's because doing so **maximizes the expected utility**
- LLMs act the same because we evaluate them as if they are taking exams
- Or, the way we train and evaluate LLMs encourages them guessing, making them hallucinate

# Coming next

# Coming next

- Hands-on practice
  - Text generation
  - Prompt engineering
  - Notebook/Solution
- Next lecture
  - Post-training adaptation
  - Course feedback and discussion
  - Guest lecture

# References I


Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. Advances in Neural Information Processing Systems, 33:1877–1901, 2020.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers), pages 4171–4186. Association for Computational Linguistics, June 2019.

Angela Fan, Mike Lewis, and Yann Dauphin. Hierarchical neural story generation. In Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 889–898, 2018.

Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. The curious case of neural text degeneration. arXiv preprint arXiv:1904.09751, 2019.

Adam Tauman Kalai, Ofir Nachum, Santosh S Vempala, and Edwin Zhang. Why language models hallucinate. arXiv preprint arXiv:2509.04664, 2025.

Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. arXiv preprint arXiv:2001.08361, 2020.

Lujun Li, Lama Sleem, Geoffrey Nichil, Radu State, et al. Exploring the impact of temperature on large language models: Hot or cold? Procedia Computer Science, 264:242–251, 2025.

Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. Advances in Neural Information Processing Systems, 35: 27730–27744, 2022.

Max Peeperkorn, Tom Kouwenhoven, Dan Brown, and Anna Jordanous. Is temperature the creativity parameter of large language models? arXiv preprint arXiv:2405.00492, 2024.